SQL - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

SQL — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

---

SAMPLE SCHEMA USED THROUGHOUT THIS DOCUMENT

Every example and practice question in this document uses this SAME consistent schema, so you can follow along and actually run these queries yourself in any RDBMS (MySQL/PostgreSQL/SQL Server).

```
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50) NOT NULL,
location VARCHAR(50)
);

CREATE TABLE employees (
emp_id INT PRIMARY KEY,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
dept_id INT,
salary DECIMAL(10,2),
hire_date DATE,
manager_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id),
FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
);

CREATE TABLE projects (
project_id INT PRIMARY KEY,
project_name VARCHAR(100) NOT NULL,
dept_id INT,
budget DECIMAL(12,2),
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

TOPIC 1: DDL (DATA DEFINITION LANGUAGE)

---

1. Introduction

What is it? DDL is the subset of SQL commands used to define/modify the STRUCTURE of database objects (tables, schemas, indexes) — `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.

Why introduced: A relational database needs a standardized way to describe what data it will hold (table structure, column types, constraints) BEFORE any actual data is inserted — DDL is that structural definition language.

2. Key DDL Commands

Command	Purpose
`CREATE`	Creates a new database object (table, index, view, schema)
`ALTER`	Modifies an EXISTING object's structure (add/drop/modify columns)
`DROP`	Permanently DELETES an object AND all its data — irreversible
`TRUNCATE`	Removes ALL ROWS from a table, but keeps the table structure itself intact
`RENAME`	Renames an existing object

3. SQL Commands — Full Examples

```
-- CREATE a table
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50) NOT NULL
);

-- ALTER: add a new column
ALTER TABLE employees ADD COLUMN email VARCHAR(100);

-- ALTER: modify a column's type
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2); -- MySQL syntax
-- ALTER TABLE employees ALTER COLUMN salary TYPE DECIMAL(12,2); -- PostgreSQL syntax

-- ALTER: drop a column
ALTER TABLE employees DROP COLUMN email;

-- ALTER: rename a column
ALTER TABLE employees RENAME COLUMN first_name TO fname; -- PostgreSQL/MySQL 8+

-- RENAME a table
ALTER TABLE employees RENAME TO staff;

-- TRUNCATE (removes ALL rows, keeps structure)
TRUNCATE TABLE employees;

-- DROP (removes the table ENTIRELY, structure and data both gone)
DROP TABLE employees;
```

4. `DROP` vs `TRUNCATE` vs `DELETE` — Critical Comparison Table

Aspect	`DROP`	`TRUNCATE`	`DELETE` (a DML command, covered next topic)
Removes structure?	Yes — the ENTIRE table object is gone	No — structure stays, only rows removed	No — structure stays, only rows removed
Removes data?	Yes	Yes — ALL rows	Depends — can remove SOME or ALL rows (via `WHERE`)
Can be rolled back?	Depends on DB/transaction support (often NOT easily)	Typically NOT logged row-by-row (harder/impossible to roll back in many systems)	YES — fully transactional, can `ROLLBACK`
WHERE clause?	Not applicable	Not allowed — always removes everything	Allowed — can target specific rows
Speed for removing ALL rows	N/A	Very FAST (deallocates data pages, doesn't log each row)	Slower (logs each row deletion)
Resets auto-increment counters?	N/A (table is gone)	Usually YES	Usually NO

5. Real Life Analogy

- **Library**: `CREATE TABLE` is like building a new labeled bookshelf. `ALTER TABLE` is like adding/removing a shelf slot or relabeling it. `TRUNCATE` is like removing ALL books from the shelf but keeping the shelf itself standing. `DROP` is like demolishing the entire shelf structure completely.

6. Edge Cases

Case	Result
`DROP TABLE` on a table referenced by a `FOREIGN KEY` elsewhere	Fails with a constraint violation error, unless the dependent table/constraint is dropped first (or `CASCADE` is used, where supported)
`TRUNCATE` on a table with active foreign key references pointing TO it	May be blocked, depending on the RDBMS and constraint configuration
Running `ALTER TABLE ... DROP COLUMN` on a column used by a `VIEW`	May fail or leave the view broken, depending on the RDBMS

7. Practice Questions (DDL)

Q1 (Easy): Write a `CREATE TABLE` statement for a `customers` table with `customer_id` (primary key), `name` (required), and `email`.

```
CREATE TABLE customers (
customer_id INT PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(100)
);
```

Q2 (Medium): Add a new `phone` column to the `employees` table, and then rename it to `contact_number`.

```
ALTER TABLE employees ADD COLUMN phone VARCHAR(15);
ALTER TABLE employees RENAME COLUMN phone TO contact_number;
```

Q3 (Hard, interview-style): Explain why `TRUNCATE TABLE employees;` is generally much faster than `DELETE FROM employees;` for removing all rows, and identify one scenario where you would be FORCED to use `DELETE` instead of `TRUNCATE`.

> **Answer**: `TRUNCATE` deallocates entire data pages at once and typically doesn't log individual row deletions, making it far faster for bulk removal; `DELETE` logs each row individually (enabling rollback) and can fire row-level triggers, making it slower. You'd be FORCED to use `DELETE` when you need to remove only SOME rows (via a `WHERE` clause) or need the operation to be fully transactional/rollback-able within a larger multi-step transaction.

8. Frequently Asked Interview Questions

1. **What does DDL stand for, and name its core commands?** Data Definition Language — `CREATE`, `ALTER`, `DROP`, `TRUNCATE`, `RENAME`.
2. **What's the key difference between `DROP` and `TRUNCATE`?** `DROP` removes the entire table structure AND its data; `TRUNCATE` removes only the data, keeping the table structure intact.
3. **Can `TRUNCATE` be rolled back?** Generally no (or with much more difficulty than `DELETE`) in most RDBMS implementations, since it typically isn't logged row-by-row.
4. **Does `TRUNCATE` support a `WHERE` clause?** No — it always removes all rows unconditionally.

TOPIC 2: DML (DATA MANIPULATION LANGUAGE)

---

1. Introduction

What is it? DML is the subset of SQL commands used to MANIPULATE the actual DATA within tables (not the structure) — `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

2. Key DML Commands with Full Examples

```
-- INSERT a single row
INSERT INTO departments (dept_id, dept_name, location)
VALUES (1, 'Engineering', 'Bangalore');

-- INSERT multiple rows in one statement
INSERT INTO departments (dept_id, dept_name, location) VALUES
(2, 'Sales', 'Mumbai'),
(3, 'HR', 'Delhi');

-- SELECT - retrieve data
SELECT first_name, last_name, salary FROM employees WHERE dept_id = 1;

-- UPDATE - modify existing rows
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 1;

-- DELETE - remove specific rows
DELETE FROM employees WHERE hire_date < '2015-01-01';
```

3. `SELECT` — The Full Anatomy

```
SELECT first_name, last_name, salary -- WHAT columns to retrieve
FROM employees -- WHICH table
WHERE dept_id = 1 -- ROW-LEVEL filter
ORDER BY salary DESC -- sort order
LIMIT 10; -- restrict result count (MySQL/Postgres);
-- SQL Server uses TOP 10 instead
```

4. Real Life Analogy

- **Restaurant**: DML is like the actual day-to-day operations of running the restaurant using its already-built kitchen (DDL structure) — placing new orders (`INSERT`), retrieving order details (`SELECT`), modifying an order (`UPDATE`), and canceling an order (`DELETE`).

5. Edge Cases

Case	Result
`UPDATE`/`DELETE` without a `WHERE` clause	Affects EVERY SINGLE ROW in the table — a classic, dangerous, easily-made mistake in production
`INSERT` violating a `NOT NULL`/`PRIMARY KEY`/`FOREIGN KEY` constraint	Fails with a constraint violation error
`SELECT` with a typo'd column name	`SQLException`/error — column doesn't exist

6. Common Mistakes

- Running `UPDATE`/`DELETE` WITHOUT a `WHERE` clause in production — one of the most common, catastrophic real-world SQL mistakes (affects ALL rows).
- Forgetting `LIMIT`/`TOP` when only testing a query against a large table, accidentally pulling millions of rows.

7. Best Practices

- ALWAYS run a `SELECT` with the SAME `WHERE` clause FIRST, to verify exactly which rows would be affected, before running the corresponding `UPDATE`/`DELETE`.
- Wrap risky `UPDATE`/`DELETE` operations in an explicit transaction (see the Transactions topic) so you can `ROLLBACK` if something looks wrong before committing.

8. Practice Questions (DML)

Q1 (Easy): Insert a new employee: id 101, "Alice", "Johnson", dept 1, salary 75000, hired 2023-01-15.

```
INSERT INTO employees (emp_id, first_name, last_name, dept_id, salary, hire_date)
VALUES (101, 'Alice', 'Johnson', 1, 75000, '2023-01-15');
```

Q2 (Medium): Give every employee in department 2 a 5% raise.

```
UPDATE employees SET salary = salary * 1.05 WHERE dept_id = 2;
```

Q3 (Hard): Delete all employees who have no department assigned (`dept_id IS NULL`) AND were hired before 2018-01-01.

```
DELETE FROM employees WHERE dept_id IS NULL AND hire_date < '2018-01-01';
```

9. Frequently Asked Interview Questions

1 **What does DML stand for, and name its core commands?** Data Manipulation Language — `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

2 **What happens if you run `UPDATE employees SET salary = 0;` with no `WHERE` clause?** EVERY row's salary is set to 0 — a common, dangerous mistake.

3 **How can you safely verify what an `UPDATE`/`DELETE` will affect before running it?** Run a `SELECT` with the IDENTICAL `WHERE` clause first to preview the affected rows.

TOPIC 3: DCL (DATA CONTROL LANGUAGE)

---

1. Introduction

What is it? DCL commands manage database ACCESS PERMISSIONS/PRIVILEGES — `GRANT` and `REVOKE`, controlling which users/roles can perform which operations on which objects.

2. Key Commands with Examples

```
-- GRANT specific privileges to a user
GRANT SELECT, INSERT ON employees TO analyst_user;

-- GRANT ALL privileges on a table
GRANT ALL PRIVILEGES ON employees TO admin_user;

-- REVOKE a previously granted privilege
REVOKE INSERT ON employees FROM analyst_user;

-- GRANT with the ability to further grant to others
GRANT SELECT ON employees TO team_lead WITH GRANT OPTION;
```

3. Real Life Analogy

- **Bank**: DCL is like the bank's internal access-control system — deciding which EMPLOYEE ROLES can VIEW customer balances (`SELECT`), which can PROCESS deposits (`INSERT`), and which can APPROVE loan modifications (`UPDATE`) — different staff get different, deliberately scoped permissions.

4. Frequently Asked Interview Questions

1. **What does DCL stand for, and name its two core commands?** Data Control Language — `GRANT`, `REVOKE`.
2. **What does `WITH GRANT OPTION` do?** Allows the user receiving the privilege to ALSO grant that same privilege to other users.
3. **Who typically executes DCL commands in a real organization?** Database administrators (DBAs), not typical application developers.

TOPIC 4: TCL (TRANSACTION CONTROL LANGUAGE)

---

1. Introduction

What is it? TCL commands manage database TRANSACTIONS — `COMMIT`, `ROLLBACK`, `SAVEPOINT` — controlling when changes become permanent versus reversible. *(Full transactional theory/ACID properties covered in the dedicated Transactions topic later in this document; this topic focuses on the specific COMMAND syntax.)*

2. Key Commands with Examples

```
BEGIN TRANSACTION; -- (or START TRANSACTION, depending on RDBMS)

UPDATE employees SET salary = salary - 5000 WHERE emp_id = 101;
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 102;

SAVEPOINT before_bonus; -- a named checkpoint WITHIN the transaction

UPDATE employees SET salary = salary + 1000 WHERE emp_id = 102;

ROLLBACK TO before_bonus; -- undo ONLY back to the savepoint, keeping earlier changes

COMMIT; -- finalize everything up to this point, PERMANENTLY
```

3. `COMMIT` vs `ROLLBACK` vs `SAVEPOINT`

Command	Effect
`COMMIT`	Permanently saves ALL changes made since the last `COMMIT`/transaction start
`ROLLBACK`	Undoes ALL changes made since the last `COMMIT`/transaction start
`ROLLBACK TO <savepoint>`	Undoes only changes made AFTER that specific named savepoint, keeping earlier changes in the transaction intact
`SAVEPOINT <name>`	Marks a named point WITHIN a transaction that you can later roll back to, WITHOUT undoing the entire transaction

4. Frequently Asked Interview Questions

1. **What does TCL stand for, and name its core commands?** Transaction Control Language — `COMMIT`, `ROLLBACK`, `SAVEPOINT`.
2. **What's the difference between `ROLLBACK` and `ROLLBACK TO <savepoint>`?** Plain `ROLLBACK` undoes the ENTIRE transaction; `ROLLBACK TO <savepoint>` undoes only changes made after that specific named point, preserving earlier changes within the same transaction.
3. **Is DCL/TCL the same as DML?** No — DML manipulates DATA (`SELECT`/`INSERT`/`UPDATE`/`DELETE`); TCL controls the TRANSACTIONAL BOUNDARIES around those DML operations.

TOPIC 5: CONSTRAINTS

---

1. Introduction

What is it? Constraints are RULES enforced by the database itself on column/table data, guaranteeing data integrity automatically — rejecting any INSERT/UPDATE that would violate them, regardless of what application code attempts.

2. The Core Constraint Types with SQL Examples

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY, -- PRIMARY KEY: unique + not null, identifies each row
email VARCHAR(100) UNIQUE, -- UNIQUE: no two rows can have the same value
first_name VARCHAR(50) NOT NULL, -- NOT NULL: value is mandatory
dept_id INT,
salary DECIMAL(10,2) CHECK (salary > 0), -- CHECK: enforces a custom business rule
status VARCHAR(20) DEFAULT 'ACTIVE', -- DEFAULT: value used if none is provided
FOREIGN KEY (dept_id) REFERENCES departments(dept_id) -- FOREIGN KEY: must reference a valid row
elsewhere
);
```

3. Constraint Types — Full Reference Table

Constraint	Purpose	Example
`PRIMARY KEY`	Uniquely identifies each row; implies `NOT NULL` + `UNIQUE`	`emp_id INT PRIMARY KEY`
`FOREIGN KEY`	Enforces that a column's value matches an existing value in ANOTHER table	`FOREIGN KEY (dept_id) REFERENCES departments(dept_id)`
`UNIQUE`	No two rows may share the same value in this column	`email VARCHAR(100) UNIQUE`
`NOT NULL`	Column must always have a value	`first_name VARCHAR(50) NOT NULL`
`CHECK`	Enforces a custom boolean condition on every row	`CHECK (salary > 0)`
`DEFAULT`	Auto-supplies a value when none is given on `INSERT`	`status VARCHAR(20) DEFAULT 'ACTIVE'`

4. Real Life Analogy

- **Bank**: Constraints are like a bank's non-negotiable account-opening rules — every account MUST have a unique account number (`PRIMARY KEY`/`UNIQUE`), a name field cannot be left blank (`NOT NULL`), and an opening deposit must be a positive amount (`CHECK`) — the bank's SYSTEM enforces these automatically, refusing any attempt to violate them, regardless of which teller/branch tries.

5. Edge Cases

Case	Result
`INSERT` violating a `CHECK` constraint (e.g., `salary = -500`)	Rejected with a constraint violation error
`INSERT` with a `dept_id` that doesn't exist in `departments`	Rejected — `FOREIGN KEY` violation
`INSERT` omitting a column with a `DEFAULT` value	Automatically uses the default, no error
Two rows attempting to share the same `UNIQUE` column value	Second `INSERT` rejected

6. Practice Questions (Constraints)

Q1 (Easy): Add a `CHECK` constraint ensuring `salary` is always greater than 0 on an EXISTING `employees` table.

```
ALTER TABLE employees ADD CONSTRAINT chk_salary_positive CHECK (salary > 0);
```

Q2 (Medium): Add a `UNIQUE` constraint on the `email` column of an existing `employees` table (assume it now has an email column).

```
ALTER TABLE employees ADD CONSTRAINT uq_email UNIQUE (email);
```

Q3 (Hard, interview-style): A `FOREIGN KEY` constraint is blocking a `DELETE` on the `departments` table because employees still reference it. Name two different SQL-level ways to resolve this, and explain the trade-off of each.

> **Answer**: (1) Delete/reassign the dependent employee rows FIRST, then delete the department — safest, most explicit. (2) Define the foreign key with `ON DELETE CASCADE` (deletes dependent employees automatically) or `ON DELETE SET NULL` (sets their `dept_id` to `NULL`) — more convenient but riskier, since `CASCADE` can silently delete large amounts of related data if not carefully considered.

7. Frequently Asked Interview Questions

1. **What does a `PRIMARY KEY` constraint imply automatically?** Both `NOT NULL` and `UNIQUE` — no two rows can share the same primary key value, and it can never be null.
2. **What's the difference between `UNIQUE` and `PRIMARY KEY`?** A table can have only ONE `PRIMARY KEY` (though composite), but MULTIPLE `UNIQUE` constraints on different columns; `UNIQUE` columns CAN allow a single `NULL` (in most RDBMS), unlike `PRIMARY KEY`.
3. **What does a `CHECK` constraint do?** Enforces a custom boolean condition that every row must satisfy (e.g., `salary > 0`).
4. **What happens if you try to insert a row referencing a non-existent foreign key value?** The `INSERT` is rejected with a foreign key constraint violation.

TOPIC 6: PRIMARY & FOREIGN KEYS

---

1. Introduction

What is it? Primary keys and foreign keys together implement RELATIONAL INTEGRITY — the mechanism that lets separate tables reference each other CORRECTLY and CONSISTENTLY, forming the "relational" in "relational database."

2. Primary Key — Deep Dive

```
-- Single-column primary key
CREATE TABLE departments (
dept_id INT PRIMARY KEY,
dept_name VARCHAR(50)
);

-- Composite (multi-column) primary key
CREATE TABLE enrollments (
student_id INT,
course_id INT,
enrolled_date DATE,
PRIMARY KEY (student_id, course_id) -- the COMBINATION must be unique, not each column alone
);
```

3. Foreign Key — Deep Dive

```
CREATE TABLE employees (
emp_id INT PRIMARY KEY,
dept_id INT,
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
ON DELETE SET NULL -- if the referenced department is deleted, set dept_id to NULL here
ON UPDATE CASCADE -- if the referenced dept_id VALUE changes, update it here automatically too
);
```

4. `ON DELETE`/`ON UPDATE` Referential Actions

Action	Effect When the Referenced Row is Deleted/Updated
`CASCADE`	Automatically deletes/updates the dependent (child) rows too
`SET NULL`	Sets the foreign key column to `NULL` in dependent rows
`RESTRICT` / `NO ACTION` (often the default)	BLOCKS the delete/update entirely if dependent rows exist
`SET DEFAULT`	Sets the foreign key column to its defined default value

5. Real Life Analogy

- **College**: `departments.dept_id` is like a department's official registration code — every `students.dept_id` (foreign key) MUST match an actually-existing department code; you cannot enroll a student in "Department 99" if no such department exists in the college's official records.

6. Edge Cases

Case	Result
Deleting a department row with `ON DELETE CASCADE` foreign keys pointing to it	ALL employees in that department are automatically deleted too — a genuinely dangerous, easily-misused feature if not deliberately intended
A composite primary key with a `NULL` in one of its columns	Not allowed — EVERY column in a composite primary key must be non-null
Self-referencing foreign key (e.g., `employees.manager_id REFERENCES employees.emp_id`)	Fully supported — models hierarchical relationships (an employee's manager IS another employee)

7. Practice Questions

Q1 (Easy): Write a `CREATE TABLE` for `orders` with a foreign key to a `customers` table.

```
CREATE TABLE orders (
order_id INT PRIMARY KEY,
customer_id INT,
order_date DATE,
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
);
```

Q2 (Medium): Modify the `employees` table's `manager_id` foreign key so that if a manager is deleted, their direct reports' `manager_id` becomes `NULL` instead of blocking the delete.

```
ALTER TABLE employees DROP CONSTRAINT fk_manager; -- (drop the existing FK first, name may vary)
ALTER TABLE employees
ADD CONSTRAINT fk_manager FOREIGN KEY (manager_id)
REFERENCES employees(emp_id) ON DELETE SET NULL;
```

Q3 (Hard): Explain why a composite primary key `(student_id, course_id)` on an `enrollments` table is more appropriate than a single auto-incrementing `enrollment_id`, in terms of enforcing a specific business rule.

> **Answer**: A composite key `(student_id, course_id)` inherently PREVENTS the same student from being enrolled in the same course twice, since that combination must be unique — this business rule is enforced automatically by the schema itself. A single auto-incrementing `enrollment_id` would allow duplicate `(student_id, course_id)` pairs unless a SEPARATE `UNIQUE` constraint were added explicitly.

8. Frequently Asked Interview Questions

1. **What is a composite primary key?** A primary key made up of MULTIPLE columns together, where the COMBINATION (not any single column alone) must be unique.
2. **What does `ON DELETE CASCADE` do?** Automatically deletes dependent (child) rows when the referenced (parent) row is deleted.
3. **What's a self-referencing foreign key, and give an example?** A foreign key column referencing the SAME table it belongs to — e.g., `employees.manager_id` referencing `employees.emp_id`, modeling a management hierarchy.
4. **Can a foreign key column contain `NULL`?** Yes (unless separately constrained with `NOT NULL`) — it simply means that row has no associated parent record for that relationship.

TOPIC 7: JOINS

---

1. Introduction

What is it? A `JOIN` combines rows from two (or more) tables based on a related column between them — the fundamental mechanism for querying ACROSS the relationships established by foreign keys.

2. The Join Types — Full Reference with SQL and Venn-Diagram-Style Explanation

INNER JOIN — only matching rows from BOTH tables

```
SELECT e.first_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
-- Returns ONLY employees who HAVE a matching department (excludes employees with a NULL/invalid
dept_id)
```

LEFT (OUTER) JOIN — ALL rows from the LEFT table, matched where possible

```
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
-- Returns EVERY employee, even those with NO matching department (dept_name will be NULL for those)
```

RIGHT (OUTER) JOIN — ALL rows from the RIGHT table, matched where possible

```
SELECT e.first_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
-- Returns EVERY department, even those with NO employees assigned (first_name will be NULL for those)
```

FULL OUTER JOIN — ALL rows from BOTH tables, matched where possible

```
SELECT e.first_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;
-- Returns EVERYTHING: matched pairs, unmatched employees, AND unmatched departments
-- (Note: MySQL has no native FULL OUTER JOIN - must simulate via UNION of LEFT and RIGHT JOIN)
```

CROSS JOIN — every row from table A paired with every row from table B (Cartesian product)

```
SELECT e.first_name, p.project_name
FROM employees e
CROSS JOIN projects p;
-- If employees has 10 rows and projects has 5 rows, this returns 50 rows (10 x 5)
```

SELF JOIN — a table joined to itself

```
SELECT emp.first_name AS employee, mgr.first_name AS manager
FROM employees emp
LEFT JOIN employees mgr ON emp.manager_id = mgr.emp_id;
-- Uses the SAME table twice (with aliases) to relate each employee to their manager
```

3. Visual Diagram (ASCII Venn-Style)

```
TABLE A TABLE B
+-----+ +-----+
| | | |
| A | INNER | B |
| | JOIN | |
+-----+ (overlap +-----+
only)

LEFT JOIN = ALL of A + overlap (unmatched B rows excluded)
RIGHT JOIN = ALL of B + overlap (unmatched A rows excluded)
FULL OUTER = ALL of A + ALL of B + overlap (nothing excluded)
```

4. Real Life Analogy

- **College**: `INNER JOIN` between `students` and `courses` (via an enrollment table) gives you only students who ARE enrolled in at least one course. `LEFT JOIN` gives you EVERY student, including those not yet enrolled in anything (showing `NULL` for their course). `CROSS JOIN` would be like generating every possible student-course PAIRING regardless of actual enrollment — rarely useful directly, but demonstrates the mathematical Cartesian product concept clearly.

5. Edge Cases

Case	Result
`INNER JOIN` where a row's join column is `NULL`	That row is EXCLUDED from the result — `NULL` never matches anything, including another `NULL`, in a join condition
Forgetting the `ON` clause entirely	In most RDBMS, this is either a syntax error (for explicit `JOIN`) or silently produces a CROSS JOIN (Cartesian product) if using old-style comma-separated `FROM table1, table2` syntax without a `WHERE` filter
`CROSS JOIN` on large tables	Can produce an ENORMOUS result set (`rows_A` × `rows_B`) — a genuine performance/memory risk if used accidentally

6. Common Mistakes

- Using old-style comma-joins (`FROM a, b WHERE a.id = b.id`) instead of explicit `JOIN ... ON` syntax — functionally similar for INNER JOIN, but far more error-prone (forgetting the `WHERE` filter accidentally creates a CROSS JOIN) and less readable.
- Confusing `LEFT JOIN` and `RIGHT JOIN` direction — remember: the table NAMED FIRST in `LEFT JOIN` is "kept in full."
- Forgetting `NULL` never matches in a join condition, causing unexpectedly excluded rows.

7. Practice Questions (Joins)

Q1 (Easy): Write a query listing every employee's name alongside their department's name.

```
SELECT e.first_name, e.last_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;
```

Q2 (Medium): List EVERY department, including those with ZERO employees, showing the department name and employee count (0 for empty departments).

```
SELECT d.dept_name, COUNT(e.emp_id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

Q3 (Hard, interview-style): Write a query listing each employee alongside their MANAGER's name (self-join), including employees who have NO manager (top-level executives).

```
SELECT emp.first_name AS employee_name, mgr.first_name AS manager_name
FROM employees emp
LEFT JOIN employees mgr ON emp.manager_id = mgr.emp_id;
```

Q4 (Hard, company-tagged: Amazon/product-based screening): Find all departments that have NO employees at all.

```
SELECT d.dept_name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
WHERE e.emp_id IS NULL;
```

8. Frequently Asked Interview Questions

1. **What's the difference between `INNER JOIN` and `LEFT JOIN`?** `INNER JOIN` returns only rows with matches in BOTH tables; `LEFT JOIN` returns ALL rows from the left table, with `NULL`s for unmatched right-table columns.
2. **What is a `CROSS JOIN`?** A Cartesian product — every row from table A paired with every row from table B, with no matching condition.
3. **What is a self-join, and give a real use case?** A table joined to itself, commonly used to model hierarchical relationships (e.g., relating each employee to their manager, who is also a row in the same `employees` table).
4. **Why doesn't `NULL = NULL` match in a join condition?** Because SQL's three-valued logic treats any comparison involving `NULL` (including `NULL = NULL`) as `UNKNOWN`, not `TRUE` — so such rows are excluded from `INNER JOIN` and matched-side results.
5. **How would you find departments with zero employees using a join?** A `LEFT JOIN` from `departments` to `employees`, filtering for `WHERE e.emp_id IS NULL` (rows with no match on the right side).

TOPIC 8: SUBQUERIES

---

1. Introduction

What is it? A subquery (or "inner query"/"nested query") is a `SELECT` statement embedded INSIDE another SQL statement — used to compute an intermediate result that the outer query then uses for filtering, comparison, or as a data source.

2. Types of Subqueries with Examples

Scalar Subquery (returns exactly ONE value)

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees); -- compares each salary to the OVERALL average
```

Multi-Row Subquery (used with `IN`, `ANY`, `ALL`)

```
SELECT first_name FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'Bangalore');
```

Correlated Subquery (references the OUTER query's current row — re-evaluated PER ROW)

```
SELECT e1.first_name, e1.salary
FROM employees e1
WHERE e1.salary > (
SELECT AVG(e2.salary)
FROM employees e2
WHERE e2.dept_id = e1.dept_id -- references e1 from the OUTER query!
);
-- Finds employees earning more than their OWN department's average (not the company-wide average)
```

Subquery in `FROM` (a "derived table")

```
SELECT dept_id, avg_salary
FROM (
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id
) AS dept_averages
WHERE avg_salary > 60000;
```

`EXISTS` *Subquery*

```
SELECT d.dept_name
FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
-- Returns departments that HAVE at least one employee (semantically similar to an INNER JOIN here,
-- but EXISTS can be more efficient - it stops at the FIRST match, doesn't need to count/return all
matches)
```

3. Correlated vs Non-Correlated Subqueries — Comparison Table

Aspect	Non-Correlated Subquery	Correlated Subquery
References outer query?	No — computed ONCE, independently	Yes — references a column from the outer query's CURRENT row
Execution	Runs ONCE total	Conceptually re-evaluated FOR EACH ROW of the outer query (though real query optimizers often rewrite this more efficiently)
Performance	Generally faster (single execution)	Can be slower on large datasets due to per-row (or per-row-equivalent) evaluation

4. Real Life Analogy

- **Bank**: A non-correlated subquery is like calculating the bank's OVERALL average account balance ONCE, then comparing every customer against that single number. A correlated subquery is like calculating EACH BRANCH's own average balance separately, and comparing each customer only against THEIR OWN branch's average — a fundamentally different, per-group comparison.

5. Edge Cases

Case	Result
A scalar subquery accidentally returning MULTIPLE rows	Error: "subquery returns more than one row"
`IN` subquery returning `NULL` among its results	Can cause surprising `NOT IN` behavior — if ANY value in the subquery's result is `NULL`, a `WHERE x NOT IN (subquery)` clause may unexpectedly return NO rows at all (a classic, dangerous SQL gotcha)

6. Common Mistakes

- Using `NOT IN` with a subquery that might return `NULL` values — a well-known gotcha that silently produces zero results; use `NOT EXISTS` instead for safety.
- Writing an unnecessarily correlated subquery when a simple `JOIN` (or `GROUP BY` with `HAVING`) would achieve the same result more efficiently.

7. Best Practices

- Prefer `NOT EXISTS` over `NOT IN` when the subquery's result might contain `NULL` values.
- Consider whether a `JOIN` could express the same logic more efficiently/clearly before reaching for a subquery, especially a correlated one.

8. Practice Questions (Subqueries)

Q1 (Easy): Find all employees earning MORE than the company-wide average salary.

```
SELECT first_name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q2 (Medium): Find employees who earn more than their OWN department's average salary (correlated subquery).

```
SELECT e1.first_name, e1.salary, e1.dept_id
FROM employees e1
WHERE e1.salary > (SELECT AVG(e2.salary) FROM employees e2 WHERE e2.dept_id = e1.dept_id);
```

Q3 (Hard, interview-style): Find departments that have NO employees, using `NOT EXISTS` (instead of the `LEFT JOIN` approach from the Joins topic).

```
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

Q4 (Hard): Explain why `WHERE dept_id NOT IN (SELECT dept_id FROM departments WHERE dept_id IS NULL)` might unexpectedly return ZERO rows, even though it looks logically correct.

> **Answer**: If the subquery returns even ONE `NULL` value among its results (here, guaranteed, since it explicitly filters FOR `NULL`), the `NOT IN` comparison against a list CONTAINING a `NULL` evaluates to `UNKNOWN` for every row, causing the ENTIRE query to return zero rows — this is a classic SQL three-valued-logic gotcha; `NOT EXISTS` avoids it entirely since it doesn't perform value-by-value list comparison the same way.

9. Frequently Asked Interview Questions

1. **What is a subquery?** A `SELECT` statement nested inside another SQL statement, used to compute an intermediate result for filtering, comparison, or as a data source.
2. **What's the difference between a correlated and non-correlated subquery?** A correlated subquery references a column from the OUTER query's current row (conceptually re-evaluated per row); a non-correlated subquery is fully independent and computed once.
3. **Why is `NOT IN` dangerous when the subquery might return `NULL`?** Because SQL's three-valued logic makes the comparison evaluate to `UNKNOWN` (never `TRUE`) whenever the list contains a `NULL`, silently causing the entire query to return zero rows.
4. **What's the safer alternative to `NOT IN` with subqueries?** `NOT EXISTS`, which doesn't suffer from the same `NULL`-related gotcha.
5. **What is a "derived table"?** A subquery used in the `FROM` clause, treated as if it were a regular (temporary, query-scoped) table for the rest of the outer query.

TOPIC 9: VIEWS

---

1. Introduction

What is it? A `VIEW` is a saved, named `SELECT` query that behaves like a VIRTUAL TABLE — it doesn't store data itself (typically), but re-runs its underlying query every time it's referenced, providing a reusable, simplified, or access-controlled window onto one or more real tables.

2. Code Examples

```
-- CREATE a view
CREATE VIEW high_earners AS
SELECT first_name, last_name, salary, dept_id
FROM employees
WHERE salary > 80000;

-- USE a view exactly like a table
SELECT * FROM high_earners WHERE dept_id = 1;

-- UPDATE via a (simple) view - works if the view is based on a SINGLE table with no aggregation
UPDATE high_earners SET salary = salary * 1.05 WHERE first_name = 'Alice';

-- DROP a view
DROP VIEW high_earners;
```

3. Why Use Views

Benefit	Explanation
Simplification	Hide a complex multi-join query behind a single, simple `SELECT * FROM view_name`
Security/Access control	Expose only SPECIFIC columns/rows to certain users (e.g., a view excluding `salary` for general staff)
Consistency	Centralize a commonly-used query definition in ONE place, instead of duplicating complex SQL everywhere

4. Materialized Views (A Related, Distinct Concept)

A regular `VIEW` re-executes its query EVERY time it's accessed (always up-to-date, but potentially slow for complex queries). A **materialized view** (supported in PostgreSQL, Oracle) PHYSICALLY STORES the query's result, refreshed periodically or manually — trading perfect freshness for significantly faster read performance on expensive aggregate queries.

```
CREATE MATERIALIZED VIEW dept_salary_summary AS
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees GROUP BY dept_id;

REFRESH MATERIALIZED VIEW dept_salary_summary; -- manually update its stored data
```

5. Real Life Analogy

- **Bank**: A regular view is like a live, always-current summary report generated fresh every time someone requests it. A materialized view is like a PRINTED daily summary report — fast to hand out, but only as current as the last time it was printed (refreshed).

6. Edge Cases

Case	Result
Attempting to `UPDATE`/`INSERT` through a view based on MULTIPLE joined tables or containing aggregation	Often disallowed or ambiguous — most RDBMS restrict updatability to simple, single-table views
The underlying table's structure changes (a column is dropped)	The view may break/error the next time it's queried, since it's re-executed against the current schema

7. Frequently Asked Interview Questions

1. **What is a SQL View?** A saved, named query that behaves like a virtual table, re-executing its underlying `SELECT` each time it's accessed.
2. **Does a View store data itself?** No (a REGULAR view doesn't) — it re-runs its query live each time; a MATERIALIZED view DOES physically store its result, refreshed periodically.
3. **Name two real reasons to use a View.** Simplifying complex/repeated queries behind a simple name, and restricting which columns/rows specific users can see (security).

4 **Can you always `UPDATE`/`INSERT` through a View?** No — only simple, single-table views without aggregation are typically updatable; views involving joins or `GROUP BY` are usually read-only.

TOPIC 10: INDEXES

---

1. Introduction

What is it? An index is an auxiliary data structure (typically a **B-Tree**, sometimes a hash structure) that dramatically speeds up row lookups on specific columns — analogous to a book's index letting you jump directly to a topic instead of reading every page sequentially.

2. Code Examples

```
-- CREATE an index on a single column
CREATE INDEX idx_employees_dept_id ON employees(dept_id);

-- CREATE a composite (multi-column) index
CREATE INDEX idx_employees_dept_salary ON employees(dept_id, salary);

-- CREATE a UNIQUE index (enforces uniqueness AND speeds up lookups)
CREATE UNIQUE INDEX idx_employees_email ON employees(email);

-- DROP an index
DROP INDEX idx_employees_dept_id;
```

3. How Indexes Work Internally (B-Tree)

```
WITHOUT an index: finding dept_id = 5 requires scanning EVERY row -> O(n) "full table scan"

WITH a B-Tree index on dept_id:
[ 10 ]
/ \
[5, 7] [15, 20]
/ | \ / \
... ... ...
-> the database navigates the TREE (O(log n)) to find matching rows directly,
instead of scanning the entire table
```

4. Trade-offs — Indexes Aren't Free

Benefit	Cost
Dramatically faster `SELECT`/`WHERE`/`JOIN`/`ORDER BY` performance on indexed columns	Extra disk space to store the index structure itself
	SLOWER `INSERT`/`UPDATE`/`DELETE`, since the index must also be updated on every write
	Too many indexes on a heavily-written table can significantly hurt write throughput

5. When to Add an Index

- Columns frequently used in `WHERE` clauses.
- Columns frequently used in `JOIN` conditions (foreign key columns are prime candidates).
- Columns frequently used in `ORDER BY`.
- **NOT** every column — indexing everything blindly hurts write performance and wastes storage without proportional read benefit.

6. Real Life Analogy

- **Library**: An index is exactly like a library's card catalog (or a book's index page) — instead of walking down every single shelf looking for a specific book (a full table scan), you consult the catalog (index) to jump DIRECTLY to the right shelf/section.

7. Edge Cases

Case	Result
An index on a column with very LOW cardinality (e.g., a `boolean` "is_active" column with only 2 possible values)	Often provides LITTLE benefit — the query optimizer may choose to ignore it and do a full scan anyway, since it doesn't narrow results much
Using a function on an indexed column in `WHERE` (e.g., `WHERE UPPER(name) = 'ALICE'`)	The plain index typically CANNOT be used efficiently — a specialized "functional index" would be needed instead
Composite index `(dept_id, salary)` used for a query filtering ONLY on `salary` (not `dept_id`)	Generally CANNOT be used efficiently — composite indexes are optimized for LEFTMOST-PREFIX queries

8. Common Mistakes

- Adding indexes on every column "just in case," hurting write performance without meaningful read benefit.
- Not indexing frequently-joined foreign key columns, causing slow joins on large tables.
- Applying a function to an indexed column in `WHERE`, unintentionally preventing the index from being used.

9. Frequently Asked Interview Questions

1. **What is a database index, and what data structure typically backs it?** An auxiliary structure that speeds up row lookups on specific columns, typically implemented as a B-Tree.
2. **What's the time complexity improvement an index provides?** From O(n) (full table scan) to roughly O(log n) for lookups on the indexed column.
3. **What's the trade-off of adding an index?** Faster reads on that column, but slower writes (`INSERT`/`UPDATE`/`DELETE`) since the index itself must also be maintained, plus additional storage space.
4. **What is the "leftmost prefix" rule for composite indexes?** A composite index on `(col1, col2)` can efficiently serve queries filtering on `col1` alone, or `col1 AND col2` together, but generally CANNOT efficiently serve a query filtering on `col2` alone.

TOPIC 11: STORED PROCEDURES

1. Introduction

What is it? A stored procedure is a precompiled, named block of SQL (and procedural logic — loops, conditionals, variables) stored and executed DIRECTLY WITHIN the database server itself, callable by name instead of sending raw SQL text each time.

2. Code Example (MySQL syntax)

```
DELIMITER //

CREATE PROCEDURE GiveRaise(IN emp_id_param INT, IN raise_percent DECIMAL(5,2))
BEGIN
UPDATE employees
SET salary = salary * (1 + raise_percent / 100)
WHERE emp_id = emp_id_param;
END //

DELIMITER ;

-- CALL the stored procedure
CALL GiveRaise(101, 10.0); -- gives employee 101 a 10% raise
```

3. A Procedure with an `OUT` Parameter

```
DELIMITER //
CREATE PROCEDURE GetEmployeeCount(IN dept_id_param INT, OUT total INT)
BEGIN
SELECT COUNT(*) INTO total FROM employees WHERE dept_id = dept_id_param;
END //
DELIMITER ;

CALL GetEmployeeCount(1, @count);
SELECT @count; -- retrieve the OUT parameter's value
```

(This connects directly to Java's `CallableStatement`, covered in the JDBC section, which is specifically designed to invoke stored procedures like this from application code.)

4. Real Life Analogy

- **Bank**: A stored procedure is like a pre-approved, standardized banking FORM the bank's own internal system processes directly — "Process Standard Loan Payment" — you just submit the required inputs, and the bank's OWN systems (not an external client application) execute the entire multi-step logic internally.

5. Stored Procedures vs Application-Level Logic — The Trade-off

Aspect	Stored Procedure (in the DB)	Application-Level Logic (in Java)
Performance	Can be faster (no round-trip for multi-step logic, precompiled)	More round-trips if logic requires multiple queries
Portability	Tied to a SPECIFIC database vendor's procedural SQL dialect	Portable across databases (standard JDBC)
Testability	Harder to unit-test in isolation	Easier — standard Java testing tools apply
Version control / code review	Often less integrated into standard application CI/CD	Fully integrated into standard development workflows
Modern trend	Used selectively for genuinely performance-critical, data-heavy operations	STRONGLY preferred for most business logic in modern application architectures

6. Frequently Asked Interview Questions

1. **What is a stored procedure?** A precompiled, named block of SQL/procedural logic stored and executed directly within the database server, callable by name.
2. **What are `IN`, `OUT`, and `INOUT` parameters in a stored procedure?** `IN` passes data INTO the procedure; `OUT` returns a value FROM the procedure back to the caller; `INOUT` does both.
3. **What's a trade-off of putting business logic in stored procedures instead of application code?** Potential performance benefits (fewer round-trips, precompiled execution) versus reduced portability, harder testability, and less integration with standard application development workflows.
4. **Which Java API is specifically designed to call stored procedures?** `CallableStatement` (JDBC).

TOPIC 12: FUNCTIONS (SQL FUNCTIONS)

---

1. Introduction

What is it? A SQL function is similar to a stored procedure, but MUST return a single value (or table, for table-valued functions) and can be used DIRECTLY within a `SELECT` statement's expression — unlike a stored procedure, which is invoked separately via `CALL`.

2. Built-in (Standard) SQL Functions — Quick Reference

Category	Examples
Aggregate	`COUNT()`, `SUM()`, `AVG()`, `MIN()`, `MAX()`
String	`CONCAT()`, `UPPER()`, `LOWER()`, `SUBSTRING()`, `TRIM()`, `LENGTH()`
Date/Time	`NOW()`, `CURDATE()`, `DATEDIFF()`, `DATE_ADD()`
Numeric	`ROUND()`, `CEIL()`, `FLOOR()`, `ABS()`
Conditional	`COALESCE()` (first non-null value), `NULLIF()`, `CASE WHEN ... THEN ... END`

```
SELECT
CONCAT(first_name, ' ', last_name) AS full_name,
ROUND(salary, 0) AS rounded_salary,
COALESCE(email, 'no-email@company.com') AS email_safe,
CASE
WHEN salary > 100000 THEN 'Senior'
WHEN salary > 60000 THEN 'Mid'
ELSE 'Junior'
END AS level
FROM employees;
```

3. User-Defined Function (UDF) Example

```
DELIMITER //
CREATE FUNCTION AnnualSalary(monthly_salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
RETURN monthly_salary * 12;
END //
DELIMITER ;

SELECT first_name, AnnualSalary(salary / 12) AS annual FROM employees;
```

4. Stored Procedure vs Function — Comparison Table

Aspect	Stored Procedure	Function
Return value	Optional (can return zero, one, or multiple via `OUT` params)	MUST return exactly one value (or a table, for table-valued functions)
Callable from `SELECT`?	No — invoked separately via `CALL`	Yes — can be used directly inside a `SELECT` expression
Can modify data (`INSERT`/`UPDATE`/`DELETE`)?	Yes	Typically restricted/discouraged (should be side-effect-free, especially if used inside `SELECT`)

5. Frequently Asked Interview Questions

1 **What's the key difference between a stored procedure and a function?** A function MUST return a single value and can be used directly inside a `SELECT` expression; a stored procedure is invoked separately via `CALL` and doesn't need to return anything.

2 **What does `COALESCE()` do?** Returns the FIRST non-null value among its arguments — commonly used to supply a fallback for potentially `NULL` columns.

3 **Should a SQL function generally modify data (perform `INSERT`/`UPDATE`)?** No — functions, especially when used inside `SELECT` statements, should generally be side-effect-free (deterministic, read-only) to avoid unexpected behavior.

TOPIC 13: TRIGGERS

---

1. Introduction

What is it? A trigger is a stored, named block of procedural SQL that AUTOMATICALLY EXECUTES in response to a specific DML event (`INSERT`, `UPDATE`, `DELETE`) on a specific table — without needing to be explicitly called, similar in spirit to a Java `Listener` (see Advanced Java Topic 7) reacting to lifecycle events.

2. Code Example — An Audit Log Trigger

```
CREATE TABLE salary_audit (
audit_id INT AUTO_INCREMENT PRIMARY KEY,
emp_id INT,
old_salary DECIMAL(10,2),
new_salary DECIMAL(10,2),
changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE TRIGGER trg_salary_update
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
IF OLD.salary <> NEW.salary THEN
INSERT INTO salary_audit (emp_id, old_salary, new_salary)
VALUES (OLD.emp_id, OLD.salary, NEW.salary);
END IF;
END //
DELIMITER ;

-- This UPDATE automatically fires the trigger above - NO application code needed to log it!
UPDATE employees SET salary = 90000 WHERE emp_id = 101;
```

3. Trigger Timing and Events

Timing	Event	Combination Example
`BEFORE`	`INSERT`/`UPDATE`/`DELETE`	Validate/modify data BEFORE it's written (e.g., auto-uppercase a column)
`AFTER`	`INSERT`/`UPDATE`/`DELETE`	React AFTER the change is made (e.g., audit logging, as above)

- `OLD.column` — the row's value BEFORE the change (available in `UPDATE`/`DELETE` triggers).

- `NEW.column` — the row's value AFTER the change (available in `INSERT`/`UPDATE` triggers).

4. Real Life Analogy

- **Bank**: A trigger is like an automatic fraud-alert system that fires the INSTANT a large withdrawal is recorded — no bank employee needs to manually check and decide to send an alert; the SYSTEM reacts automatically to the specific database event itself.

5. Edge Cases

Case	Result
A trigger that itself performs an `UPDATE` on the SAME table it's triggered by	Can cause infinite recursion/cascading triggers if not carefully designed — many RDBMS have recursion limits or require explicit configuration to allow this safely
Multiple triggers on the SAME event for the SAME table	Execute in an order that may be RDBMS-specific/configurable, not always guaranteed

6. Common Mistakes

- Overusing triggers for complex business logic that would be clearer and more maintainable in application code — "invisible" logic hidden in database triggers is notoriously hard to discover/debug for developers unfamiliar with the schema.
- Creating triggers with unintended recursive side effects.

7. Best Practices

- Use triggers primarily for genuinely cross-cutting, data-integrity-focused concerns (audit logging, enforcing complex constraints beyond `CHECK`) — not general business logic.
- Document triggers clearly, since their execution is invisible/implicit from application code's perspective.

8. Frequently Asked Interview Questions

1. **What is a database trigger?** A stored block of SQL that automatically executes in response to a specific `INSERT`/`UPDATE`/`DELETE` event on a table, without needing explicit invocation.
2. **What's the difference between `OLD` and `NEW` in a trigger?** `OLD` refers to the row's values BEFORE the change; `NEW` refers to its values AFTER the change.
3. **What's a real, common use case for triggers?** Audit logging — automatically recording old/new values whenever a sensitive column (like salary) changes.
4. **Why should triggers be used sparingly for business logic?** Because their execution is invisible/implicit to anyone reading application code, making the system's behavior harder to discover, understand, and debug compared to explicit application-level logic.

TOPIC 14: WINDOW FUNCTIONS

1. Introduction

What is it? Window functions perform calculations ACROSS a set of rows related to the current row (a "window"), WITHOUT collapsing the result into a single row per group — unlike `GROUP BY` aggregates, which DO collapse multiple rows into one summary row per group.

2. The Critical Distinction: Window Functions vs `GROUP BY`

```
-- GROUP BY: COLLAPSES rows - one row PER department, individual employee rows are LOST
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept_id;

-- WINDOW FUNCTION: KEEPS every individual row, ADDS a computed column alongside it
SELECT first_name, dept_id, salary,
AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg_salary
FROM employees;
-- Every employee row is STILL present, each showing their OWN department's average ALONGSIDE their
own salary
```

3. Key Window Functions

```
SELECT
first_name, dept_id, salary,
RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS salary_rank,
DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rank,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,
LAG(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS prev_hire_salary,
LEAD(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS next_hire_salary
FROM employees;
```

Function	Purpose
`ROW_NUMBER()`	Assigns a UNIQUE sequential number (1, 2, 3, ...) to each row within its partition
`RANK()`	Assigns a rank, with TIES sharing the same rank, but LEAVING GAPS afterward (1, 1, 3, 4)
`DENSE_RANK()`	Assigns a rank, with TIES sharing the same rank, but NO GAPS afterward (1, 1, 2, 3)
`LAG(col)`	Accesses a PREVIOUS row's value (relative to the current row, per the `ORDER BY`)
`LEAD(col)`	Accesses the NEXT row's value
`SUM()/AVG()/COUNT() OVER (...)`	Aggregate functions used WITHOUT collapsing rows

4. `PARTITION BY` — Grouping Without Collapsing

`PARTITION BY dept_id` divides rows into GROUPS (like `GROUP BY` would), but the window function computes its result PER PARTITION while still returning EVERY individual row — the defining characteristic that makes window functions so powerful for "rank within group" and "compare to group average" style queries.

5. Real Life Analogy

- **College**: A window function is like ranking EVERY student's exam score WITHIN THEIR OWN CLASS, while still listing every individual student's row — unlike `GROUP BY`, which would collapse the entire class down to just one summary row (like the class average), losing each individual student's row entirely.

6. Edge Cases

Case	Result
`RANK()` vs `DENSE_RANK()` with tied values	`RANK()` skips subsequent rank numbers after a tie (1, 1, 3); `DENSE_RANK()` does not (1, 1, 2)
`LAG()`/`LEAD()` on the FIRST/LAST row of a partition (no previous/next row exists)	Returns `NULL` by default (a second argument can supply a custom default instead)
Using an aggregate window function WITHOUT `PARTITION BY`	Treats the ENTIRE result set as one single partition (e.g., a running company-wide average alongside every row)

7. Practice Questions (Window Functions)

Q1 (Medium): Rank employees by salary WITHIN their own department (highest salary = rank 1).

```
SELECT first_name, dept_id, salary,
RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_salary_rank
FROM employees;
```

Q2 (Hard, interview-style): Find the TOP 2 highest-paid employees in EACH department.

```
SELECT * FROM (
SELECT first_name, dept_id, salary,
ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
FROM employees
) ranked
WHERE rn <= 2;
```

Q3 (Hard, company-tagged: analytics/product-based screening): For each employee, show their salary alongside the NEXT-hired employee's salary in the same department (using `LEAD`).

```
SELECT first_name, dept_id, hire_date, salary,
LEAD(salary) OVER (PARTITION BY dept_id ORDER BY hire_date) AS next_hire_salary
FROM employees;
```

8. Frequently Asked Interview Questions

1 **What's the key difference between a window function and `GROUP BY`?** `GROUP BY` collapses multiple rows into one summary row per group; window functions compute a value per group (via `PARTITION BY`) WITHOUT collapsing — every individual row is preserved.

2 **What's the difference between `RANK()` and `DENSE_RANK()`?** Both assign the SAME rank to tied values, but `RANK()` leaves GAPS in subsequent rank numbers after a tie, while `DENSE_RANK()` does not.

3 **What does `PARTITION BY` do in a window function?** Divides the result set into groups (like `GROUP BY`), with the window function computing its result independently per group, while still returning every individual row.

4 **How would you find the top N rows per group using a window function?** Use `ROW_NUMBER() OVER (PARTITION BY group_col ORDER BY sort_col DESC)` in a subquery/CTE, then filter the outer query for `rn <= N`.

TOPIC 15: CTE (COMMON TABLE EXPRESSIONS)

---

1. Introduction

What is it? A CTE (`WITH name AS (...)`) defines a NAMED, temporary result set that exists only for the duration of the single query that follows it — improving readability for complex queries by breaking them into clearly-named, logical steps, and enabling RECURSIVE queries (impossible with plain subqueries).

2. Basic (Non-Recursive) CTE

```
WITH high_earners AS (
SELECT emp_id, first_name, salary, dept_id
FROM employees
WHERE salary > 80000
)
SELECT h.first_name, d.dept_name
FROM high_earners h
JOIN departments d ON h.dept_id = d.dept_id;
```

> Functionally similar to a subquery in `FROM`, but far more READABLE for complex, multi-step queries — you can even define MULTIPLE CTEs, each referencing earlier ones, before the final `SELECT`.

3. Recursive CTE — Modeling Hierarchies (A Genuinely Unique Capability)

```
WITH RECURSIVE employee_hierarchy AS (
-- ANCHOR MEMBER: the starting point (top-level executives, no manager)
SELECT emp_id, first_name, manager_id, 1 AS level
FROM employees
WHERE manager_id IS NULL

UNION ALL

-- RECURSIVE MEMBER: joins back to the CTE itself, one level deeper each iteration
SELECT e.emp_id, e.first_name, e.manager_id, eh.level + 1
FROM employees e
INNER JOIN employee_hierarchy eh ON e.manager_id = eh.emp_id
)
SELECT * FROM employee_hierarchy ORDER BY level;
```

This solves a problem that's GENUINELY difficult or impossible to express with plain subqueries or joins alone: traversing an arbitrarily-deep hierarchy (org charts, category trees, bill-of-materials structures) in a SINGLE query.

4. Real Life Analogy

- **Bank**: A recursive CTE is like tracing a corporate ownership chain — "Company A owns Company B, which owns Company C, which owns Company D" — you don't know in advance how many levels deep the chain goes, and a recursive CTE can walk through ALL of them automatically, however deep, in one query.

5. Edge Cases

Case	Result
A recursive CTE with no proper termination condition (e.g., a circular manager reference)	Can loop indefinitely — most RDBMS enforce a maximum recursion depth to prevent this from hanging forever
Multiple CTEs referencing each other in the SAME `WITH` clause	Fully supported — later CTEs can reference earlier ones defined in the same statement

6. Common Mistakes

- Attempting to model a hierarchy (org chart, category tree) using repeated self-joins with a FIXED number of levels — brittle and breaks for any hierarchy deeper than anticipated; a recursive CTE handles ARBITRARY depth correctly.
- Forgetting the `RECURSIVE` keyword (required in PostgreSQL/standard SQL; MySQL 8+ also requires it) when writing a recursive CTE.

7. Practice Questions (CTE)

Q1 (Medium): Rewrite the "employees earning more than their department average" query (from the Subqueries topic) using a CTE instead of a correlated subquery.

```
WITH dept_avg AS (
SELECT dept_id, AVG(salary) AS avg_salary
FROM employees GROUP BY dept_id
)
SELECT e.first_name, e.salary, d.avg_salary
FROM employees e
JOIN dept_avg d ON e.dept_id = d.dept_id
WHERE e.salary > d.avg_salary;
```

Q2 (Hard, interview-style): Write a recursive CTE to find ALL employees who report (directly or indirectly, through any number of management levels) to a specific manager (emp_id = 1).

```
WITH RECURSIVE reports AS (
SELECT emp_id, first_name, manager_id
FROM employees WHERE manager_id = 1

UNION ALL

SELECT e.emp_id, e.first_name, e.manager_id
FROM employees e
```

```
INNER JOIN reports r ON e.manager_id = r.emp_id
)
SELECT * FROM reports;
```

8. Frequently Asked Interview Questions

1. **What is a CTE?** A named, temporary result set (`WITH name AS (...)`) that exists only for the duration of the single following query, improving readability for complex multi-step queries.
2. **What can a recursive CTE do that a plain subquery/join cannot?** Traverse an arbitrarily-deep hierarchical structure (org charts, category trees) in a single query, without knowing the depth in advance.
3. **What are the two parts of a recursive CTE?** The "anchor member" (the starting point/base case) and the "recursive member" (which references the CTE itself, joining one level deeper each iteration), combined via `UNION ALL`.
4. **What prevents a recursive CTE from running forever on a circular reference?** Most RDBMS enforce a maximum recursion depth limit as a safety mechanism.

TOPIC 16: TRANSACTIONS (SQL PERSPECTIVE)

1. Introduction

(The ACID properties and general transaction theory are covered in full in the JDBC section's Transactions topic from a Java/application perspective — this topic focuses specifically on the pure SQL-level commands and behavior, usable directly at a database console/tool, independent of any application language.)

2. Full SQL Transaction Example

```
START TRANSACTION;

UPDATE employees SET salary = salary - 5000 WHERE emp_id = 101;
UPDATE employees SET salary = salary + 5000 WHERE emp_id = 102;

-- Check the result BEFORE committing (still visible only within THIS transaction/session)
SELECT emp_id, salary FROM employees WHERE emp_id IN (101, 102);

COMMIT; -- makes it PERMANENT and visible to all other sessions
-- or: ROLLBACK; -- undoes both UPDATEs entirely
```

3. Isolation Levels — SQL Syntax

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
START TRANSACTION;
-- ... statements ...
COMMIT;
```

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
`READ UNCOMMITTED`	Possible	Possible	Possible
`READ COMMITTED` (common default)	Prevented	Possible	Possible
`REPEATABLE READ`	Prevented	Prevented	Possible
`SERIALIZABLE`	Prevented	Prevented	Prevented

- **Dirty read**: reading another transaction's UNCOMMITTED changes.
- **Non-repeatable read**: re-reading the SAME row within one transaction gives a DIFFERENT result because another transaction committed a change in between.
- **Phantom read**: re-running the SAME query within one transaction returns a DIFFERENT SET OF ROWS because another transaction inserted/deleted matching rows in between.

4. Locking (Brief SQL-Level Overview)

```
SELECT * FROM employees WHERE emp_id = 101 FOR UPDATE; -- explicitly locks the selected row(s)
-- until the current transaction commits/rolls back
```

`FOR UPDATE` prevents OTHER transactions from modifying (or, depending on the RDBMS, even reading) those specific rows until the current transaction finishes — used to safely implement "read, then modify based on what you read" logic without a race condition between two concurrent transactions.

5. Frequently Asked Interview Questions

1. **What SQL command starts an explicit transaction?** `START TRANSACTION` (or `BEGIN`, depending on the RDBMS).
2. **What's a "dirty read"?** Reading data that another transaction has changed but NOT yet committed — if that other transaction later rolls back, you'd have read data that never actually "existed" permanently.
3. **What's the difference between a non-repeatable read and a phantom read?** A non-repeatable read is re-reading the SAME specific row and getting a different VALUE; a phantom read is re-running the SAME query and getting a different SET OF ROWS (due to inserts/deletes by another transaction).
4. **What does `SELECT ... FOR UPDATE` do?** Explicitly locks the selected rows, preventing other transactions from modifying (or sometimes even reading) them until the current transaction completes.

TOPIC 17: QUERY OPTIMIZATION

---

1. Introduction

What is it? Query optimization is the practice of writing and structuring SQL so the database engine can execute it as efficiently as possible — understanding HOW the database actually processes a query (via its query planner/optimizer) to avoid common, expensive mistakes.

2. `EXPLAIN` — Seeing the Query Execution Plan

```
EXPLAIN SELECT * FROM employees WHERE dept_id = 1;
-- Shows whether the database will use an INDEX SCAN (fast) or a FULL TABLE SCAN (slow)
-- for this specific query, along with estimated row counts and cost
```

3. Common Optimization Techniques with Examples

```
-- BAD: SELECT * pulls ALL columns, even unused ones - wastes I/O and network bandwidth
SELECT * FROM employees WHERE dept_id = 1;

-- GOOD: select only what you actually need
SELECT first_name, last_name FROM employees WHERE dept_id = 1;

-- BAD: applying a function to an indexed column PREVENTS the index from being used efficiently
SELECT * FROM employees WHERE YEAR(hire_date) = 2023;

-- GOOD: rewrite as a RANGE condition, which CAN use an index on hire_date
SELECT * FROM employees WHERE hire_date >= '2023-01-01' AND hire_date < '2024-01-01';

-- BAD: a correlated subquery that could be an efficient JOIN instead
SELECT * FROM employees e WHERE dept_id IN (SELECT dept_id FROM departments WHERE location =
'Bangalore');

-- OFTEN BETTER (verify with EXPLAIN for your specific database/data):
SELECT e.* FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
WHERE d.location = 'Bangalore';
```

4. Key Principles

Principle	Why It Matters
Index frequently filtered/joined columns	Turns O(n) full scans into O(log n) index lookups (see the Indexes topic)
Avoid `SELECT *`	Reduces unnecessary I/O and network transfer, especially with wide tables
Avoid functions on indexed columns in `WHERE`	Preserves the database's ability to actually USE the index
Use `LIMIT`/`TOP` when you don't need all results	Avoids computing/transferring unnecessary rows
Prefer `EXISTS` over `IN` for large subquery results	Can short-circuit at the first match instead of materializing the full list
Analyze with `EXPLAIN` before assuming	Query optimizers can behave counter-intuitively — MEASURE, don't guess

5. Real Life Analogy

- **Library**: Query optimization is like choosing the FASTEST way to find a specific book — using the catalog/index (an actual database index) instead of walking every single shelf (full table scan), and not requesting the ENTIRE library's inventory list (`SELECT *`) when you only need ONE book's details.

6. Common Mistakes

- Using `SELECT *` habitually, even in production application queries.
- Applying functions to columns inside `WHERE` clauses, unknowingly disabling index usage.
- Never checking `EXPLAIN` output before assuming a query is "fine" simply because it returns correct results — CORRECTNESS and PERFORMANCE are separate concerns.

7. Frequently Asked Interview Questions

1. **What does the `EXPLAIN` command show?** The database's actual EXECUTION PLAN for a query — whether it uses an index scan or full table scan, estimated costs, and row counts.
2. **Why does `WHERE YEAR(hire_date) = 2023` typically perform worse than a range comparison?** Applying a function to the indexed column prevents the database from using a standard index efficiently, often forcing a full table scan instead.
3. **Why should you avoid `SELECT *` in production queries?** It retrieves unnecessary columns, wasting I/O, memory, and network bandwidth, especially on wide tables.
4. **When might `EXISTS` outperform `IN` for a subquery?** When the subquery's result set is large — `EXISTS` can short-circuit at the FIRST match, while `IN` may need to materialize the entire result list first.

TOPIC 18: NORMALIZATION

---

1. Introduction

What is it? Normalization is the process of organizing database tables to MINIMIZE DATA REDUNDANCY and avoid update/insert/delete ANOMALIES, by progressively decomposing tables according to a series of formal rules called **Normal Forms (1NF, 2NF, 3NF, ...)**.

Why introduced: A poorly-designed, "flat" table (e.g., storing an employee's department NAME repeated in every single row, instead of referencing a separate `departments` table) wastes storage AND creates dangerous inconsistency risks — updating a department's name would require updating potentially THOUSANDS of duplicate rows, and missing even one creates silently inconsistent data.

2. The Problem — An Unnormalized Table

```
UNNORMALIZED "employees" table:
+--------+-------+------------+-----------------+
| emp_id | name | dept_name | dept_location |
+--------+-------+------------+-----------------+
| 1 | Alice | Engineering| Bangalore |
| 2 | Bob | Engineering| Bangalore | <- "Engineering"/"Bangalore" DUPLICATED
| 3 | Carol | Sales | Mumbai |
+--------+-------+------------+-----------------+

PROBLEMS:
- UPDATE anomaly: renaming "Engineering" requires updating MULTIPLE rows consistently
- INSERT anomaly: can't add a new department until at least one employee is assigned to it
- DELETE anomaly: deleting the only employee in "Sales" ACCIDENTALLY loses ALL information
about the "Sales" department itself
```

3. First Normal Form (1NF) — Atomic Values, No Repeating Groups

```
VIOLATES 1NF (multiple phone numbers crammed into one column):
+--------+-------+-----------------------+
| emp_id | name | phone_numbers |
+--------+-------+-----------------------+
| 1 | Alice | "9876543210, 9123456789"|
+--------+-------+-----------------------+

1NF-COMPLIANT (separate table, one phone number per row):
employees: emp_id, name
employee_phones: emp_id, phone_number
```

Rule: Every column must hold a single, ATOMIC value — no comma-separated lists, no repeating groups within one row.

4. Second Normal Form (2NF) — No Partial Dependency (on a Composite Key)

```
VIOLATES 2NF (assumes a composite key: student_id + course_id):
+------------+-----------+--------------+--------------+
| student_id | course_id | course_name | student_name |
+------------+-----------+--------------+--------------+
-- course_name depends ONLY on course_id (part of the key), NOT the full composite key
-- student_name depends ONLY on student_id (part of the key), NOT the full composite key

2NF-COMPLIANT (split into three properly-scoped tables):
students: student_id, student_name
courses: course_id, course_name
enrollments: student_id, course_id, enrollment_date
```

Rule: (Applies to tables with a COMPOSITE primary key) Every non-key column must depend on the ENTIRE composite key, not just PART of it.

5. Third Normal Form (3NF) — No Transitive Dependency

```
VIOLATES 3NF:
+--------+---------+-----------+----------------+
| emp_id | name | dept_id | dept_location |
+--------+---------+-----------+----------------+
-- dept_location depends on dept_id, NOT directly on emp_id (the primary key) -
-- this is a TRANSITIVE dependency: emp_id -> dept_id -> dept_location

3NF-COMPLIANT (matches our SAMPLE SCHEMA from the top of this document!):
employees: emp_id, name, dept_id
departments: dept_id, dept_location
```

Rule: Every non-key column must depend DIRECTLY on the primary key, not on ANOTHER non-key column.

6. Real Life Analogy

- **College**: An unnormalized `students` table storing the full department NAME and LOCATION repeated for every single student is like writing a student's entire department's address by hand on EVERY individual student record — normalizing means each student record just references a department CODE, with the department's actual details stored ONCE in a separate `departments` table, exactly like our sample schema throughout this document.

7. Normalization vs Denormalization — A Real Trade-off

Aspect	Normalized	Denormalized (intentionally, for performance)
Data redundancy	Minimized	Some duplication accepted
Update anomalies	Prevented	More risk of inconsistency
Read performance (complex queries needing many joins)	Can require MORE joins, potentially slower	Fewer joins needed, can be FASTER for read-heavy analytical workloads
Typical use	Transactional (OLTP) systems, correctness-critical data	Data warehouses/reporting (OLAP) systems, read-heavy analytics

> **Real-world nuance**: Modern systems often deliberately DENORMALIZE specific tables for performance (e.g., a reporting table that duplicates data for fast read access) — normalization is the DEFAULT correct starting point, but not an absolute, unbreakable rule in every single scenario.

8. Practice Questions (Normalization)

Q1 (Medium): Given a single flat table `orders(order_id, customer_name, customer_email, product_name, product_price, quantity)`, identify which normal form(s) it violates and how you'd redesign it.

> **Answer**: It violates 3NF (and arguably 2NF depending on the key) — `customer_name`/`customer_email` depend on the customer, not directly on `order_id`; `product_name`/`product_price` depend on the product, not directly on `order_id`. Redesign into: `customers(customer_id, name, email)`, `products(product_id, name, price)`, `orders(order_id, customer_id, product_id, quantity)`.

Q2 (Hard, interview-style): Explain a real scenario where you might DELIBERATELY denormalize a table, and what risk you're accepting by doing so.

Answer: A reporting/analytics dashboard querying millions of rows might denormalize by storing a pre-joined, duplicated `dept_name` directly in a reporting table, avoiding an expensive JOIN on every dashboard load. The risk accepted is DATA INCONSISTENCY if the source department name changes and the denormalized copy isn't properly kept in sync (requiring a deliberate refresh/ETL process).

9. Frequently Asked Interview Questions

1. **What is normalization, and why is it done?** Organizing tables to minimize data redundancy and prevent update/insert/delete anomalies, via a series of formal rules (Normal Forms).
2. **What does 1NF require?** Every column must hold a single, atomic value — no repeating groups or comma-separated lists within one column.
3. **What does 2NF add on top of 1NF?** No PARTIAL dependency — every non-key column must depend on the ENTIRE composite primary key, not just part of it (only relevant for tables with composite keys).
4. **What does 3NF add on top of 2NF?** No TRANSITIVE dependency — every non-key column must depend DIRECTLY on the primary key, not on another non-key column.
5. **When might you deliberately denormalize a table?** For read-heavy analytical/reporting workloads where join performance matters more than eliminating redundancy — accepting some data-consistency risk in exchange for faster reads.

FINAL SUMMARY — SQL COMPLETE

This concludes the **SQL** section — 18 topics, all with runnable SQL commands and practice questions against a consistent sample schema:

1. **DDL** — `CREATE`/`ALTER`/`DROP`/`TRUNCATE`, and the critical `DROP` vs `TRUNCATE` vs `DELETE` distinction.
2. **DML** — `SELECT`/`INSERT`/`UPDATE`/`DELETE`, and the danger of missing `WHERE` clauses.
3. **DCL** — `GRANT`/`REVOKE` access control.
4. **TCL** — `COMMIT`/`ROLLBACK`/`SAVEPOINT` transaction command syntax.
5. **Constraints** — `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, `CHECK`, `DEFAULT`.
6. **Primary & Foreign Keys** — composite keys, `ON DELETE`/`ON UPDATE` referential actions.
7. **Joins** — `INNER`/`LEFT`/`RIGHT`/`FULL OUTER`/`CROSS`/`SELF` joins with full examples.
8. **Subqueries** — scalar, multi-row, correlated, `EXISTS`, and the `NOT IN` NULL gotcha.
9. **Views** — virtual tables, materialized views.
10. **Indexes** — B-Trees, the leftmost-prefix rule, read/write trade-offs.
11. **Stored Procedures** — `IN`/`OUT`/`INOUT` parameters, ties to JDBC's `CallableStatement`.
12. **Functions** — built-in and user-defined, vs stored procedures.
13. **Triggers** — `BEFORE`/`AFTER`, `OLD`/`NEW`, audit-logging patterns.
14. **Window Functions** — `RANK()`/`DENSE_RANK()`/`ROW_NUMBER()`/`LAG()`/`LEAD()`, `PARTITION BY` vs `GROUP BY`.
15. **CTE** — named subqueries, recursive CTEs for hierarchy traversal.

16 **Transactions** — isolation levels, dirty/non-repeatable/phantom reads, `FOR UPDATE` locking.

17 **Query Optimization** — `EXPLAIN`, index-aware query writing, common anti-patterns.

18 **Normalization** — 1NF/2NF/3NF with worked examples, and when to deliberately denormalize.

You now have a complete, practical SQL reference with runnable commands and interview-ready practice questions — the direct foundation for the PostgreSQL, Hibernate, and Spring Data JPA sections ahead.

(END OF SQL MASTER NOTES — ready to proceed to PostgreSQL whenever you say "Next Topic.")